

Week 6: Pre-Class Study Guide - Neural Networks & Keras

Session: Week 6 - Neural Network Fundamentals with Keras **Time Required:** 30 minutes **Purpose:** Prime your brain for neural networks before live session

Overview

This week, you're transitioning from traditional ML (sklearn) to deep learning (Keras). Neural networks are powerful but conceptually different. This pre-class work builds intuition BEFORE we dive into code.

Philosophy: Concepts first, math never (for this course).

You'll understand WHAT neural networks do and WHY they work, without needing to derive backpropagation equations. Focus on visual intuition.

Required Videos (50 minutes total)

Video 1: But what is a Neural Network? (19 min)

Watch: 3Blue1Brown - But what is a Neural Network?

What you'll learn:

- Visual walkthrough of a simple neural network recognizing handwritten digits (MNIST)
- How neurons connect and pass information forward
- What weights and biases actually do
- How activation functions introduce non-linearity
- Intuition for how layers learn hierarchical features

Key takeaway: Neural networks learn by adjusting weights to recognize patterns, layer by layer.

What to focus on:

- The structure: input layer $\rightarrow$ hidden layers $\rightarrow$ output layer
- How neurons compute: weighted sum + bias + activation function
- How deeper layers learn more abstract features (edges $\rightarrow$ shapes $\rightarrow$ digits)

What to skip:

- Mathematical derivations (we won't need them)
 - Calculus notation (3Blue1Brown shows it for completeness, but you don't need to memorize)
-

Video 2: Gradient Descent - How Neural Networks Learn (21 min)

Watch: 3Blue1Brown - Gradient Descent, how neural networks learn

What you'll learn:

- How networks adjust weights to minimize error (loss)
- Visual intuition for gradient descent (rolling down a hill)
- Why networks need many training examples
- What backpropagation does conceptually (don't worry about the math!)

Key takeaway: Networks "learn" by iteratively adjusting weights in the direction that reduces error.

What to focus on:

- Gradient descent as "finding the valley" in the loss landscape

- Why we need many iterations (epochs) to reach good weights
- The idea of backpropagation: error flows backward through layers

What to skip:

- Chain rule calculus (we won't implement backpropagation from scratch)
 - Partial derivative notation (Keras handles this for us)
-

Interactive Exploration: TensorFlow Playground (10 min)

Explore: TensorFlow Playground

What to do:

1. **Start simple:**
 - Choose “Circle” dataset (top-left)
 - Use 1 hidden layer with 4 neurons
 - Click “Play” () and watch the network learn!
2. **Observe patterns:**
 - How does the decision boundary change over epochs?
 - What happens if you add more neurons?
 - What happens if you add another hidden layer?
3. **Try spiral dataset:**
 - Select “Spiral” (harder problem)
 - Start with 1 hidden layer, 4 neurons
 - Click play - does it solve the problem?
 - Now add a second hidden layer with 4 neurons
 - Click play again - does it improve?

Key takeaway: Deeper networks (more layers) can solve more complex problems. But they also need more data and training time.

Questions to ponder:

- Why does the spiral dataset need more layers than the circle dataset?
 - What do the colors in the visualization represent?
 - How do you know when the network has “converged” (finished learning)?
-

Key Concepts to Understand Before Class

1. Neural Network Architecture

What is it? A neural network is a series of layers that transform input data into output predictions.

Structure:

Input Layer → Hidden Layer(s) → Output Layer

Example (MNIST digit classification):

784 pixels → 128 neurons → 64 neurons → 10 outputs (digits 0-9)

Think of it as: - Input layer: Raw features (e.g., pixel values) - Hidden layers: Learned feature detectors (edges, shapes, patterns) - Output layer: Final decision (classification probabilities)

2. Neurons and Activations

What does a neuron do?

1. Takes inputs (from previous layer)
2. Multiplies each input by a weight
3. Adds a bias
4. Passes result through an activation function
5. Sends output to next layer

Activation functions (key ones for this week):

Activation	When to Use	What It Does
ReLU	Hidden layers	$\max(0, x)$ - Simple, effective, prevents vanishing gradients
Sigmoid	Binary classification output	Squashes to $[0, 1]$ - gives probability
Softmax	Multi-class classification output	Converts to probabilities that sum to 1

Key insight: Without activation functions, stacking layers is pointless (just creates complicated linear function). Activations introduce non-linearity, allowing networks to learn complex patterns.

3. Training a Neural Network

The process:

1. **Forward pass:** Input flows through network, produces prediction
2. **Calculate loss:** How wrong is the prediction?
3. **Backward pass (backpropagation):** Compute how to adjust weights
4. **Update weights:** Nudge weights to reduce loss
5. **Repeat** for many epochs (passes through data)

Key terms:

- **Epoch:** One complete pass through training data
- **Batch size:** Number of samples processed before updating weights
- **Loss function:** Measures prediction error (what we minimize)
- **Optimizer:** Algorithm that updates weights (e.g., Adam, SGD)

Don't worry: Keras handles steps 2-4 automatically when you call `model.fit()`!

4. Keras Workflow (Preview)

You'll use this pattern in class:

```
# 1. Build model
model = keras.Sequential([
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])

# 2. Compile (configure training)
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

```
# 3. Train
model.fit(X_train, y_train, epochs=10)
```

```
# 4. Evaluate
model.evaluate(X_test, y_test)
```

Same pattern as sklearn! Build → Train (fit) → Evaluate. Just with more configuration steps.

Pre-Class Questions to Ponder

Think about these before class (no need to write answers, just mental prep):

1. **Why do neural networks need normalization?**
 - Hint: Pixel values are 0-255, but networks work better with 0-1. Why?
 2. **Why use multiple hidden layers instead of one very wide layer?**
 - Hint: Think about hierarchical feature learning (edges → shapes → objects)
 3. **How is a neural network different from logistic regression?**
 - Hint: Logistic regression is actually a neural network with no hidden layers!
 4. **Why do we need validation data during training?**
 - Hint: How do we know if the network is overfitting?
 5. **When would you use ReLU vs Sigmoid activation?**
 - Hint: One for hidden layers, one for output layer
-

What to Focus On (Concepts, Not Math!)

DO focus on:

- Visual intuition (how networks transform data)
- Architecture choices (layers, neurons, activations)
- The training process (epochs, loss decreasing)
- When to use neural networks vs traditional ML

DON'T worry about:

- Calculus or matrix derivatives
- Implementing backpropagation from scratch
- Mathematical proofs of why gradient descent works
- Complex optimization theory

Remember: You're learning to USE neural networks for practical applications, not to write research papers on them. Keras abstracts the math - focus on intuition and application.

Preparation Checklist

Before live session, you should:

- ☐ Watch 3Blue1Brown video 1 (neural network structure)
- ☐ Watch 3Blue1Brown video 2 (gradient descent)

- ☐ Explore TensorFlow Playground (play with circle and spiral datasets)
- ☐ Review key concepts in this guide
- ☐ Think about the pre-class questions
- ☐ Ensure Keras 3.x is installed (see technical setup guide)
- ☐ Complete pre-class practice notebook (optional but recommended)

How to verify you're ready:

- Can you explain what a neuron does (weighted sum + bias + activation)?
- Can you describe the training process (forward pass → loss → backprop → update)?
- Do you understand why we use activation functions?
- Are you comfortable with the idea of layers learning hierarchical features?

If you can answer “yes” to these, you're ready!

Technical Setup (Do This Before Class!)

Verify Keras 3.x installation:

```
import os
os.environ['KERAS_BACKEND'] = 'torch'

import keras
print(f"Keras version: {keras.__version__}")
print(f"Backend: {keras.backend.backend()}")
```

Expected output:

```
Keras version: 3.x.x
Backend: torch
```

If this doesn't work: - See KERAS_TECHNICAL_SETUP.md in week6 folder - Install: `pip install keras>=3.0 torch` - Ask for help in discussion forum BEFORE class

Optional: Bonus Exploration

If you finish early and want to dive deeper:

Read: Keras Documentation

- Keras - Introduction to Keras for Engineers
- Focus on Sequential API section
- Don't worry about Functional API yet (Week 7-8)

Play: More TensorFlow Playground Experiments

- Try different activation functions (ReLU vs Tanh vs Sigmoid)
- Try different learning rates
- Observe when networks fail to converge

Watch: Optional Videos

- StatQuest - Neural Networks Part 1 (Foundations)
 - Keras in 10 Minutes (Quick API overview)
-

Common Questions

Q: “Do I need to understand the math?”

A: Not for this course! You need to understand WHAT neural networks do (learn patterns from data) and HOW to use them (Keras API). The math is abstracted away by Keras.

Think of it like driving a car: You don’t need to know thermodynamics to drive, just how to use the gas, brake, and steering wheel.

Q: “What if I don’t understand backpropagation?”

A: That’s fine! Backpropagation is the algorithm Keras uses to update weights. You need to know it EXISTS (error flows backward) and what it ACHIEVES (better weights), but not how to derive it mathematically.

Focus on: “Backpropagation adjusts weights based on how wrong the predictions were.”

Q: “How do I choose the number of layers and neurons?”

A: Experimentation! Rules of thumb:

- Start simple (1-2 hidden layers, moderate neurons like 64-128)
- If underfitting (low accuracy): Add layers or neurons
- If overfitting (train accuracy high, test low): Reduce complexity or add regularization (Week 7)

In class, we’ll practice this experimentation!

Q: “Why Keras instead of raw PyTorch?”

A: Keras 3.x is a high-level API that runs on PyTorch backend. You get:

- Simple, intuitive API (easier to learn)
- PyTorch power under the hood (ecosystem compatibility)
- Less boilerplate code (focus on concepts, not loops)

Later, if you want low-level control, you can transition to raw PyTorch. Keras first = faster learning!

Q: “When should I use neural networks instead of XGBoost?”

A: Good question! Preview for Week 8:

- **Use neural networks for:** Images, text, audio, video (structured spatial/sequential data)
- **Use XGBoost for:** Tabular data with mixed feature types
- **Why?** XGBoost is often better for tabular data, easier to tune, faster to train

This week: Focus on learning neural networks. Week 8: Learn when to choose each.

You’re Ready!

By completing this pre-class work, you:

- Understand neural network structure (layers, neurons, activations)
- Have intuition for how networks learn (gradient descent, backpropagation)

- Explored the TensorFlow Playground (hands-on visual learning)
- Know what to expect in class (Keras API, MNIST classifier)

In the live session, you'll:

1. Build a neural network using Keras Sequential API
 2. Train on MNIST dataset (handwritten digits)
 3. Watch training curves and interpret results
 4. Experiment with different architectures (pair programming)
 5. Gain confidence to build your own networks
-

Summary

What you learned:

- Neural networks learn hierarchical features through layers
- Neurons compute: weighted sum + bias + activation
- Networks train by minimizing loss via gradient descent
- Keras makes building networks simple (Sequential API)

What you practiced:

- Visual intuition (3Blue1Brown videos)
- Interactive exploration (TensorFlow Playground)

What's next:

- Live session: Build your first MNIST classifier!
 - Pair programming: Experiment with architectures
 - Post-class: Build from scratch with guidance
-

Questions Before Class?

If anything is unclear, don't stress! The live session will bring it all together.

See you in class!

Come ready to build, experiment, and watch neural networks learn before your eyes!

Last Updated: 2026-02-02